NEAT: Semantic Prefix Analysis for Sound Constrained Code Generation with Large Language Models

Paul Kronlund-Drouault

École Normale Supérieure de Lyon, France
paul.kronlund-drouault@ens-lyon.fr

Abstract. Large language models are increasingly used as probabilistic search engines for code generation, theorem proving, and structured tool use. Existing constrained decoding systems provide strong guarantees for regular or context-free output formats, but they do not directly account for context-dependent semantic constraints such as scope, type compatibility, declaration effects, or language-specific well-formedness predicates. We introduce semantic prefix grammars (SPG), a declarative framework that reduces context-aware constrained generation to an online static analysis problem. An SPG pairs context-free productions with binding annotations and semantic premises over a decidable, abstract domain which is syntactically realizable in the attached grammar. A token is admissible exactly when the combined syntactic and semantic state remains extendable to at least one complete semantically valid expression. This yields a formal soundness invariant: every oracle-accepted prefix has a valid completion. We implement SPG in Aufbau, a Rust prefix parser based on Earley-style saturation with typed obligation stores and right-frontier context flow, and expose the oracle through P7, a constrained generation layer for Hugging Face Transformers. Across eight model families from 124M to 27B parameters, constrained mixed-mode generation improves global pass rate from 23% to 42%, with a 27B model under P7 surpassing GPT-5.4-mini on the same evaluation. These results suggest that semantic constraints can substantially improve the reliability of code generation, especially when deploying smaller open-weight models.

Keywords: Static Analysis · Constrained Decoding · Large Language Models · Type Checking · Program Synthesis

1 Introduction

Problem. Large language models are effective samplers for formal artifacts, but unconstrained sampling gives no structural guarantee: a model can always emit a token that leaves the target language, violates a type rule, or invalidates a proof script. Training, retrieval, and prompting reduce the probability of such failures, but they do not remove the failure states from the decoding process. For formal methods workloads this distinction matters: a nearly-correct decoder is still an untrusted component.

Constrained decoding addresses this problem by masking or rejecting inadmissible continuations. The best-developed line of work targets regular languages, JSON schemas, and context-free grammars. This is necessary but not sufficient for programming languages and proof-oriented languages. Many important constraints are semantic in the ordinary programming-language sense: identifiers must be in scope, declarations modify future contexts, branches must agree on types, and expression forms may be valid only under language-specific side conditions. These constraints are context-dependent even when the surface syntax is context-free.

Main idea. Building upon grammar-constrained decoding systems such as Outlines [20], subword-aligned decoding [1,12], Earley-optimized structured decoding [17], and recent type-constrained generation [8], we propose a declarative framework for prefix decoding with semantic constraints. A specification contains ordinary productions with binding annotations, plus inference-style semantic rules. The parser maintains a partial forest and a finite semantic state. A continuation is accepted when syntactic (parsing) and semantic evaluation can derive a closed work in the language by completing it.

We reuse the standard view of attribute grammars [6] and constraint/logic grammars [13,16]: a context-free derivation is accepted only when attached semantic information admits a satisfying valuation. We name our class *semantic prefix grammars* and add the condition that the constraint domain must be decidable and its partial evaluator must be realizable over prefixes (semantic completability corresponds to syntactic completability). The current instantiation supports typed languages with finite contexts, rule-local unification, binding evidence, and context transforms. Keeping the same model, richer fragments of languages such as TypeScript or Python can be obtained by extending the premise language and context structures while preserving the same parser architecture.

Contributions. This paper makes the following contributions:

1. A formal model of *prefix closure* parsing for our grammar class along with proof targets for generalized realizable semantic constraints.
2. A working implementation: a Rust completion engine (Aufbau) and a Python sampling layer (P7) that integrates with Hugging Face Transformers [21].
3. A compact prefix parsing algorithm based on Earley-style saturation [4], enriched with semantic obligations, context flow, and an end-of-input closure phase.
4. Empirical evaluation: constrained generation on zero-shot code generation with small models, and on mixed Chain-of-Thought [19] plus formal block reasoning tasks.

Target This framework can be a step toward mechanized correctness guarantees for LLM-driven program synthesis: the proof obligations are deliberately structured for eventual formalization in a proof assistant, and the framework extends to richer type systems on the same parser architecture. Deeper integration with the model layer could also lead to very interesting results, especially as a bridge between information theory (LLM distribution entropy) and formal languages.

2 Semantic Prefix Grammars

2.1 Completability

Let $L \subseteq \Sigma^*$ be a formal language over a character alphabet Σ. A prefix $s \in \Sigma^*$ is **completable** for L when

$$\exists s' \in \Sigma^*,\ ss' \in L.$$

2.2 Grammar

A Semantic Prefix Grammar is a tuple

$$G = (N, T, P, S, \Theta, \mathcal{T}, \mathcal{B}, A)$$

where:

- N is a finite set of non-terminals
- T is a finite set of terminal recognizers over segment text
- P is a finite set of productions
- $S \in N$ is the distinguished start symbol
- Θ is a finite set of semantic rules
- $\mathcal{T} : N \rightharpoonup \Theta$ maps semantic non-terminals to their rule
- $\mathcal{B}$ is the set of binding identifiers
- $A : N \to P^*$ maps each non-terminal n to its ordered list of productions

$$A(n) = (p_1^n, \ldots, p_{k_n}^n).$$

Assumption 1. *An SPG has no structural productivity constraints. We will consider the unproductive case trivial in terms of syntactic and semantic constraints, and from now on assume we are dealing with only productive grammars in the SPG class. Thus every reachable alternative in the syntactic projection has some terminal yield.*

Productions and Alternatives

Definition 1 (Production). *A* production $p_a^n \in A(n)$ *is a finite sequence*

$$p_a^n = \alpha_1^a[b_1^a]\, \alpha_2^a[b_2^a] \cdots \alpha_m^a[b_m^a],$$

where:

- $a \in \{1, \ldots, k_n\}$ *is the index of the alternative;*
- *each* $\alpha_i^a \in T \cup N$ *is either a terminal recognizer or a non-terminal;*
- *each* $b_i^a \in \mathcal{B} \cup \{\varepsilon\}$ *is either a binding identifier or the empty binding.*

The production p_a^n *is attached to the unique non-terminal* n*. We write* $|p_a^n| = m$ *for its length.*

Bindings are syntactic annotations on child positions. They don't affect the context-free structure but they name child evidence that may be referenced by semantic rules.

Segmentation. Parsing is parameterized by a segmentation policy

$$\mathsf{seg} : \Sigma^* \to \mathsf{Seg}^*.$$

We require seg to be *deterministic and prefix-aware*: an unfinished final segment may be marked open, but the segmentation of already closed interior segments is stable under extension. The segmentation algorithm is a greedy left-to-right longest-match policy. Delimiters are chosen depending on the task, but independently of the grammar.

Terminal matching is abstracted through a segment-level interface

$$\mathsf{match}(t, x) \to \{\bot\} \cup \{\mathsf{Prefix}, \mathsf{Exact}, \mathsf{Extensible}\} \times \mathsf{Reg}$$

This interface is justified by RegEx derivatives [2,11] for regular terminals. The parser itself only depends on the evidence exposed by match. The value $\bot$ means that the segment cannot match the terminal.

$$\mathsf{Term}(x, \xi, r), \qquad \xi \in \{\mathsf{Prefix}, \mathsf{Exact}, \mathsf{Extensible}\} \qquad r = \partial_x t$$

Here Reg is the set of regular residual recognizers. Terminal evidence is treated uniformly with child evidence: it may satisfy bindings, may remain open at the right frontier, and may be refined by later input. Here r is the derivative of terminal recognizer t after consuming x. If the derivative is nonempty, the current terminal evidence still has a completion.

Binding Signatures Semantic rules refer to children of a production through binding identifiers. We restrict bindings so that every semantic non-terminal has a fixed binding interface shared by all of its alternatives.

Definition 2 (Binding signature).
The binding signature *for $n \in N$ is the set $\mathcal{B}(n) \subseteq \mathcal{B}$. A grammar is* binding-regular *if for every $b \in \mathcal{B}(n)$, b appears only in productions of n and exactly once in every production.*

$$\forall n \in N,\ \forall b \in \mathcal{B}(n), \quad \Big(\forall a \in \{1, \cdots, k_n\}, \exists! i \in \{1, \cdots, |p_a^n|\}.\ b_i^a = b\Big) \\ \wedge \Big(\forall n' \in N \setminus \{n\}, \forall a' \in \{1, \cdots, k_{n'}\},\ \forall j \in \{1, \cdots, |p_{a'}^{n'}|\},\ b_j^{a'} \neq b\Big)$$

Definition 3 (Binding position). *Let G be binding-regular. For $n \in N$, $p_a^n \in A(n)$, and $b \in \mathcal{B}(n)$, define*

$$\mathsf{pos}_{n,a}(b) = \textit{the unique } i \in \{1, \ldots, |p_a^n|\} \textit{ such that } b_i^a = b.$$

This function is well-defined by totality and uniqueness of binding signatures.

The parser uses $\mathsf{pos}_{n,a}$ to route semantic obligations to the corresponding child. If an item is currently positioned before child i of alternative p_a^n, then the only binding obligations projected to that child are those for bindings b satisfying

$$\mathsf{pos}_{n,a}(b) = i.$$

Property 1. By definition of pos, and using *binding regularity*, we can establish binding positions are unique :

$$b_1 \neq b_2 \iff \mathsf{pos}_{n,a}(b_1) \neq \mathsf{pos}_{n,a}(b_2)$$

When referring to an SPG in the case of a proof or a definition, we assume binding regularity.

Syntactic state We define a syntactic state that holds our constraint structure, in order to prove its correctness.

Definition 4 (Syntactic State). *For an input $s \in \Sigma^*$, the* syntactic state *is a pair*

$$\sigma(s) = (s, t)$$

where t is an AST over G. We inductively define t as either

- *a* terminal leaf $\mathsf{Term}(x, \xi, r)$ *with $x \in \mathsf{Seg}$, $\xi \in \{\mathsf{Prefix}, \mathsf{Exact}, \mathsf{Extensible}\}$, and $r \in \Sigma^*$ a witnessing extension;*
- *an* internal node $\langle n, p, \chi \rangle$ *where $n \in N$, $p = \alpha_1^a[b_1^a] \cdots \alpha_m^a[b_m^a] \in A(n)$, and $\chi = (c_1, \ldots, c_k)$ with $k \leq m$ is a list of children such that for each $l \in \{1, \ldots, k\}$, c_l is an AST whose root recognizes α_l^a.*

The yield of t — the concatenation of segment text of terminal leaves in left-to-right order — must be consistent with $\mathsf{seg}(s)$.

The status of an AST node is propagated bottom-up:

- a terminal leaf $\mathsf{Term}(x, \xi, r)$ has status ξ;
- an internal node $\langle n, p, \chi \rangle$ with $\chi = (c_1, \ldots, c_k)$ and $|p| = m$ has status
  - Exact if $k = m$ and c_k is Exact,
  - Extensible if $k = m$ and c_k is Extensible,
  - Prefix otherwise.

The status of $\sigma(s)$ is the status of the root of t. An Exact state has been fully consumed; an Extensible state is fully consumed but its rightmost terminal could still match further input; a Prefix state has its open frontier on the rightmost descendant chain of t.

Evidence summary and binding resolution So far the syntactic state exposes only the AST t. We attach to every internal AST node a single *evidence summary*, the canonical node form used throughout the paper: the abstract constraint graph (§2.3) relates such summaries, and the parser arena (§4.2) materializes them.

Definition 5 (Evidence summary). *The summary of an internal AST node $\langle n,\ p_a^n,\ \chi \rangle$ at segment span $[i, j)$ is the tuple*

$$\nu \ = \ \langle n,\ [i, j),\ \rho, \tau, \nabla,\ \beta \rangle,$$

where $\rho \in \{\mathsf{Exact}, \mathsf{Prefix}, \mathsf{Extensible}\}$ is the propagated status, and β is the binding-resolution map of Definition 6 below. It also carries resolved constrained information in τ and exported modifications (i.e context operations) in ∇ instatiated by the constraint domain (§2.3, §3). A terminal leaf has summary $\mathsf{Term}(x, \xi, r)$ as in §2.2 and is treated as a degenerate ν with $\rho = \xi$.

Definition 6 (Frontier-aware binding resolution). *Let ν summarize $\langle n,\ p_a^n,\ (c_1, \ldots, c_k) \rangle$, and fix $b \in \mathcal{B}(n)$ with $i_b = \mathsf{pos}_{n,a}(b)$ (§2.2). The* binding resolution *of b at ν is*

$$\beta(\nu, b) \ = \ \begin{cases} \mathsf{Resolved}(\nu_{c_{i_b}}) & i_b \leq k, \\ \mathsf{Pending}(i_b) & i_b > k. \end{cases}$$

$\mathsf{Pending}(i_b)$ *records that the right frontier k has not yet reached the binding position of b, so no child evidence is available. As k advances on the same item,* $\mathsf{Pending}(i_b)$ *may transition to* Resolved*, never the reverse.*

The split Resolved/Pending is a concrete instance of the *available/unavailable* attribute distinction of incremental attribute grammars defined by Demers et al. [3]: an unavailable instance in their works corresponds here to a binding whose position lies past the dot, and its eventual *stabilization* is what the parser obligation store achieves online. The convention also matches the typed-hole formalism of Hazel [10,22], in which unfilled positions are first-class members of the static state. The Pending verdict is our parser-time analog of a Hazel hole at a binding-reference vertex.

2.3 Semantic graphs

We define a semantic layer that records information as a finite *relational graph* over the evidence summaries of §2.2. Constraints operate on a partial (prefix) tree, but the semantic relations themselves do not need to be tree-shaped. Rules relate evidence summaries directly, for example by equality, lookup, unification, subtyping, or more complex decidable predicates. This is the prefix-conservative reading of the *compound dependency graph* of attribute grammars [6,3]: vertices are attribute (evidence) instances and edges are semantic dependencies.

Definition 7 (Evidence graph). *Let $s \in \Sigma^*$ be an input prefix such that $\sigma(s)$ is reachable. The* evidence graph *at s is a finite hypergraph $\mathcal{G}(s) = (\mathcal{E}(s), \mathcal{C}(s))$ where:*

- *$\mathcal{E}(s)$ is the finite set of evidence vertices at s. Each vertex is either an evidence summary $\nu = \langle n, [i, j), \rho, \tau, \nabla, \beta \rangle$ (Definition 5), a terminal leaf $\mathsf{Term}(x, \xi, r)$, or a binding-reference $\beta(\nu, b)$ in the sense of Definition 6;*
- *$\mathcal{C}(s) \subseteq \mathcal{E}(s) \times \mathcal{E}(s)$ is the finite set of constraint edges emitted so far, drawn from a decidable constraint language and interpreted over $\mathcal{E}(s)$.*

Open and closed graphs. The open / closed split is *derived* from frontier evidence rather than asserted in prose. Following the available / unavailable attribute distinction of [3], a vertex is *frontier* exactly when it still carries some incomplete-input signal:

$$\begin{aligned} \mathsf{frontier}(\nu) &\iff \rho(\nu) \in \{\mathsf{Prefix}, \mathsf{Extensible}\}, \\ \mathsf{frontier}(\beta(\nu, b)) &\iff \beta(\nu, b) = \mathsf{Pending}(\cdot), \\ \mathsf{frontier}(\mathsf{Term}(x, \xi, r)) &\iff \xi \in \{\mathsf{Prefix}, \mathsf{Extensible}\}. \end{aligned}$$

By construction, we can assert that only one child can be at the frontier for a given prefix state node.

Definition 8 (Open / closed). *$\mathcal{G}(s)$ is* open *iff some constraint edge $c \in \mathcal{C}(s)$ is incident to a frontier vertex. $\mathcal{G}(s)$ is* closed *iff it is not open and contains no contradiction in the underlying constraint domain.*

This semantic notion of closure is distinct from language-level acceptance: a closed evidence graph may still sit inside an incomplete syntactic projection, and (under productivity, Assumption 1) every open graph admits an extension that closes it.

Example 1. In a typed lambda grammar, the prefix $\lambda x : \mathsf{Int}$ already exposes evidence for the binder x and for its type annotation, but the body binding is still $\mathsf{Pending}$ in the sense of Definition 6: the dot of the lambda item has not yet reached the body position. By Definition 8 the graph is therefore *open*, the constraint edge asserting the body type being incident to a $\mathsf{Pending}$ vertex. After reading a complete annotation and body such as $\lambda x : \mathsf{Int}.\, x$, every binding-reference becomes $\mathsf{Resolved}$ and the corresponding evidence graph is closed.

Definition 9 (Witness). *Let $\mathcal{G}(s) = (\mathcal{E}(s), \mathcal{C}(s))$ be the evidence graph at a reachable input prefix s. A* witness *for $\mathcal{G}(s)$ is a string $r \in \Sigma^*$ such that*

$$\mathcal{G}(s \cdot r) \text{ is closed.}$$

That is, extending the input by r drives the evidence graph into a closed satisfying state. The empty string ε is a witness exactly when $\mathcal{G}(s)$ is already closed.

Definition 10 (Denotation). *The* denotation *of $\mathcal{G}(s)$ is the set of its witnesses,*

$$\mathcal{D}(\mathcal{G}(s)) \;=\; \big\{\, r \in \Sigma^* \;\big|\; \mathcal{G}(s \cdot r) \text{ is closed} \,\big\}.$$

In every realizable instance, there exists a constraint-preserving morphism from $\mathcal{G}(s)$ into a closed semantic graph corresponds to a concrete string witness, and conversely each witness r induces such a morphism through the parser state reached at $s \cdot r$.

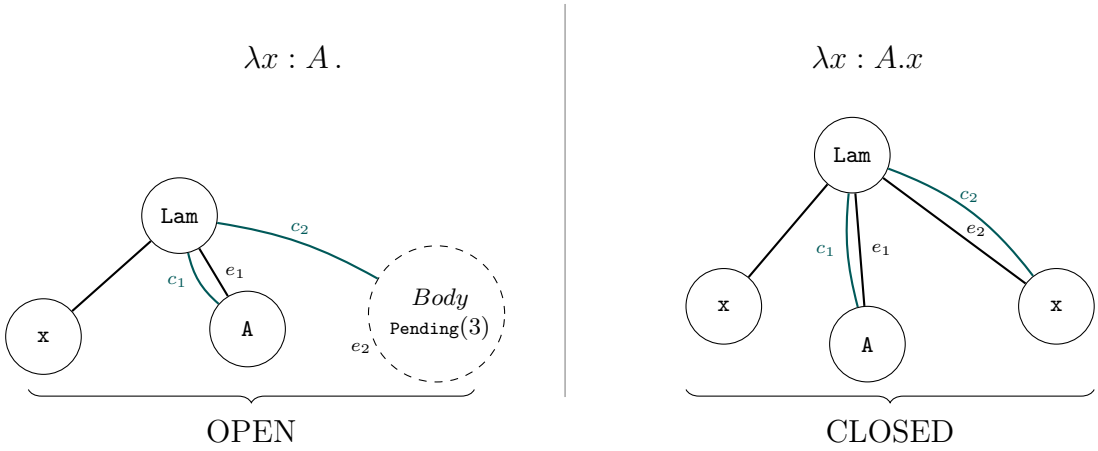

Fig. 1. Open and closed semantic graphs for prefix lambda expressions. Solid black edges are AST edges; colored curved edges are semantic links.

Definition 11 (Constraint domain). *A* constraint domain *is a triple*

$$D = (\mathsf{Rules}, \mathsf{Closed}, \mathit{eval})$$

where:

- Rules *is a decidable language of relational rules over evidence nodes and domain constants;*
- Closed *is the class of closed evidence graphs admitted by the domain;*
- *eval is the ideal evaluator,*

$$\mathit{eval}(\mathcal{G}(s)) \in \{\mathsf{Satisfied}, \mathsf{Live}, \mathsf{Lost}\}.$$

It is defined by the following cases:

$$\mathit{eval}(\mathcal{G}(s)) = \mathsf{Satisfied} \iff \mathcal{G}(s) \in \mathsf{Closed}$$

$$\mathit{eval}(\mathcal{G}(s)) = \mathsf{Live} \iff \mathcal{G}(s) \notin \mathsf{Closed} \ \wedge \ \mathcal{D}(\mathcal{G}(s)) \neq \emptyset$$

The evaluator returns

$$\mathit{eval}(\mathcal{G}(s)) = \mathsf{Lost}$$

otherwise, meaning the denotation is empty.

Lemma 1 (Safe pruning). *If the parser discards an input prefix s only when*

$$\mathit{eval}(\mathcal{G}(s)) = \mathsf{Lost},$$

then it never discards a prefix that has a closed satisfying continuation.

Proof. Suppose the parser discards s. By assumption,

$$\mathit{eval}(\mathcal{G}(s)) = \mathsf{Lost}.$$

By Definition 11 we have $\mathcal{D}(\mathcal{G}(s)) = \emptyset$: the prefix s admits no witness. Hence discarding s cannot remove a prefix that could still be completed into a closed satisfying graph.

We write $\mathsf{SPG}(D)$ for grammars over a constraint domain D equipped with the evaluator of Definition 11. Grammars in this class are said to be realizable, as their semantic constraints match their syntactic realization possibilities.

3 Simple Typing Domain Implementation

This section instantiates a constraint domain of Definition 11 with a type-directed semantic algebra. The key property of this instantiation is that the ideal evaluator of Definition 11 is *computable* given our set of rules. These rules are evaluated by $\mathit{eval}_{\mathrm{impl}}$ over an abstract syntax tree t.

Closed Types and Type Evidence

A *closed type* is a fully resolved type:

$$\tau \ ::= \ t \ \mid \ \tau_1 \to \tau_2 \ \mid \ \tau_1 \vee \tau_2 \ \mid \ \neg\tau \ \mid \ \top \ \mid \ \bot.$$

Closed-type equality and inclusion are assumed decidable, with $\bot \subseteq \tau \subseteq \top$ for all τ. A *typing context* is a finite partial map $\Gamma : \mathit{Name} \rightharpoonup \mathit{Type}$. Type information flows through the parser in three stages:

$$\text{type expression} \xrightarrow{\text{elab}} \text{type evidence} \xrightarrow{\text{close}} \text{closed type}.$$

Type evidence is the semantic state of a type-bearing evidence node:

$$E \ ::= \ \mathsf{Exact}(\tau) \ \mid \ \mathsf{Partial}(P)$$

where $P \neq \emptyset$ is a finite set of closed-type completions. *Type expressions* $\hat{\tau}$ in typing rules are elaborated to evidence:

$$\begin{aligned} \tau &\mapsto \mathsf{Exact}(\tau), \\ \Gamma(x) &\mapsto \text{the evidence stored for } x \text{ in } \Gamma, \\ \texttt{typeof}(v) &\mapsto \text{ the resolved type for node bound to } v \\ b &\mapsto \text{the evidence stored for binding } b, \\ ?A &\mapsto \text{a pattern matching on unified binding evidence} \end{aligned}$$

Meta resolution Meta-variables are rule-local and do not escape the rule application. Repeated occurrences of $?A$ within a rule are compiled into equality constraints between the corresponding nodes in the evidence graph. They serve both as pattern matching expressions, and unification shorthands for `typeof` calls. For example $b : ?A \to ?B$ means that the resolved type for binding b must be an arrow type. Adding a later constraint $a : ?B$ would create be compiled to $\texttt{typeof}(b)[1] =$ `typeof(a)` where $\tau[1]$ is the second component of the arrow type.

Binding resolution Resolving bindings uses the binding position 2.2 defined above. Every constraint on a binding b is made into an edge linking to the node referred to by b. The status of that edge is read off $\beta(\nu, b)$ (Definition 6): a Resolved entry exposes the child summary directly, while a Pending entry leaves the edge incident to a frontier vertex and the host graph open in the sense of Definition 8.

Primitive Premises

The constraint instances of Definition 7 are drawn from the following decidable premise language:

$x \in \Gamma$	name x is present in the context,
$\hat{\tau}_1 = \hat{\tau}_2$	type equality,
$\hat{\tau}_1 \subseteq \hat{\tau}_2$	type inclusion,
$\Gamma \vdash b : \hat{\tau}$	child b has type $\hat{\tau}$,
$\Gamma[x : \hat{\tau}] \vdash b : \hat{\sigma}$	child b is checked under an extended context.

Each premise is evaluated by a computable implementation of *eval* (Definition 11). We write this implementation $eval_{\text{impl}}$; Theorem 1 establishes that it coincides with the ideal evaluator on every reachable state.

Typing Rules

A typing rule $\gamma \in \Theta$ is a tuple $(name,\ M,\ Bind,\ \Phi,\ \hat{\tau}_c)$ where $name \in N$ is the attached non-terminal, M is a finite set of rule-local meta-variables, $Bind$ is the set of bindings used, $\Phi = \varphi_1, \ldots, \varphi_n$ is the ordered premise list, and $\hat{\tau}_c$ is the conclusion type expression. Rules are written in natural-deduction style:

$$\frac{\varphi_1 \quad \cdots \quad \varphi_n}{\hat{\tau}_c}\ (name).$$

Premises are evaluated left to right: a meta-variable solved by an earlier premise may appear in later premises and in the conclusion. The conclusion $\hat{\tau}_c$ is itself a type expression in the elaboration sense above. Three cases matter for the proofs below:

- a *literal* closed type (e.g. $\hat{\tau}_c = \mathsf{Int}$), which elaborates directly to $\mathsf{Exact}(\tau_c)$;
- a *context lookup* $\Gamma(x)$, which forwards the closed type bound to x in Γ;
- a *binding reference* Which is what introduces $\mathsf{Partial}$ evidence on the parent summary and drives the recursion in Lemma 4.

The ideal evaluator of Definition 11 operates on graphs; the typing-domain implementation evaluates a rule pointwise from its premises:

$$eval_{\text{impl}}(\gamma) = \begin{cases} \mathsf{Satisfied} & \forall \phi \in \Phi,\ eval_{\text{impl}}(\phi) = \mathsf{Satisfied}, \\ \mathsf{Live} & (\forall \phi \in \Phi,\ eval_{\text{impl}}(\phi) \in \{\mathsf{Satisfied}, \mathsf{Live}\}) \\ & \quad \wedge\ (\exists \phi \in \Phi,\ eval_{\text{impl}}(\phi) = \mathsf{Live}), \\ \mathsf{Lost} & \exists \phi \in \Phi,\ eval_{\text{impl}}(\phi) = \mathsf{Lost}. \end{cases}$$

Graph-level lifting. The verdict at one rule lifts to the verdict at a whole sub-evidence-graph by combining the rule's verdict with the verdicts at the node's children. Order verdicts by $\mathsf{Satisfied} < \mathsf{Live} < \mathsf{Lost}$ and write $\sqcup$ for the join (the maximum). For an internal AST node $\nu = \langle n, p_a^n, \chi \rangle$,

$$eval_{\text{impl}}(\nu) \ =\ \widehat{\mathcal{T}}(n) \ \sqcup \bigsqcup_{c \in \chi} eval_{\text{impl}}(c), \qquad \widehat{\mathcal{T}}(n) = \begin{cases} eval_{\text{impl}}(\mathcal{T}(n)) & \mathcal{T}(n) \text{ defined}, \\ \mathsf{Satisfied} & \text{otherwise}, \end{cases}$$

with the convention $\bigsqcup_{c\in\emptyset} = \mathsf{Satisfied}$ for childless nodes. For a terminal leaf,

$$eval_{\mathrm{impl}}(\mathsf{Term}(x,\xi,r)) = \begin{cases} \mathsf{Satisfied} & \xi = \mathsf{Exact}, \\ \mathsf{Live} & \xi \in \{\mathsf{Prefix}, \mathsf{Extensible}\}. \end{cases}$$

We write $eval_{\mathrm{impl}}(\mathcal{G}(s))$ for the verdict at the root of the AST in $\sigma(s)$. The join is associative and absorbs Lost, so a single Lost leaf forces the whole subtree to Lost, while Live propagates to every ancestor that is otherwise Satisfied.

Realizability

A premise on which $eval_{\mathrm{impl}}$ does not return Lost should be witnessed by an explicit input extension: a string r such that the premise is Satisfied at $\sigma(s \cdot r)$. The argument has the same skeleton everywhere. Type evidence is monotone, so the search space of completions only shrinks (Lemma 2). For premises that mention a binding b, the parser must drive every Pending binding to Resolved and recursively expose its type as Exact; this is the inductive content of `typeof` realizability (Lemma 4). Every other primitive premise is a corollary of these two facts, and the rule-level realizability of a full $\gamma \in \Theta$ (Lemma 5) follows by composing them across the premise list.

Lemma 2 (Type evidence is monotone). *Let e be an evidence node and let $\mathrm{comp}(e,s)$ denote the set of closed types that the type evidence of e at state $\sigma(s)$ can resolve to. For any extension $s \cdot r$,*

$$\mathrm{comp}(e,\ s \cdot r) \ \subseteq\ \mathrm{comp}(e,\ s).$$

Proof. By case analysis on the evidence form.

- $\mathsf{Exact}(\tau)$: comp $= \{\tau\}$, unchanged by any extension since exact evidence carries no open frontier.
- $\mathsf{Partial}(P)$: P is the finite set of closed types reachable as completions of the current type prefix. Input extension applies the regex derivative to the ongoing type segment, and derivatives can only remove completions [11], so $\mathrm{comp}(e, s \cdot r) \subseteq P$.

Remark 1. By contrapositive, monotonicity yields stability of Lost verdicts: once no completion can satisfy a premise, no further input can reverse this. The realizability lemmas below establish the dual property for the non-Lost verdicts.

Lemma 3 (Type evidence is realizable). *For any evidence node e and any $\tau \in \mathrm{comp}(e,s)$, there exists $r \in \Sigma^*$ such that the parser at state $\sigma(s \cdot r)$ records e as $\mathsf{Exact}(\tau)$.*

Proof. Trivial for Exact ($r = \varepsilon$). For $\mathsf{Partial}(P)$ with $\tau \in P$, regex-derivative completeness [11] guarantees a string r that drives the residual recognizer to acceptance with denotation τ.

Lemma 4 (`typeof` is realizable). *Let ν summarize a parent item and fix $b \in \mathcal{B}(n)$ with $i_b = \mathsf{pos}_{n,a}(b)$. Write $\texttt{typeof}_\nu(b)$ for the conclusion of the typing rule attached to the child bound to b: when $\beta(\nu,b) = \mathsf{Resolved}(\nu_c)$, $\texttt{typeof}_\nu(b) = \nu_c.\tau$, where $\nu_c.\tau$ is the elaboration of $\hat{\tau}_c$ for $\mathcal{T}(n_c) = (name_c, M_c, Bind_c, \Phi_c, \hat{\tau}_c)$ at ν_c (Typing Rules subsection above); when $\beta(\nu,b) = \mathsf{Pending}(\cdot)$, $\texttt{typeof}_\nu(b)$ is undefined. If $eval_{\mathrm{impl}}$ does not return Lost on the premise $\texttt{typeof}_\nu(b) = \hat{\tau}$ at $\sigma(s)$, there exists $r \in \Sigma^*$ such that the premise is Satisfied at $\sigma(s \cdot r)$.*

Proof. We induct on the size of the AST sub-tree rooted at the binding target c_{i_b} of ν (finite by Definition 5).

Case $\beta(\nu, b) = \mathsf{Pending}(i_b)$. The dot has not yet reached position i_b, so no child evidence is available. By productivity (Assumption 1) every reachable alternative has a terminal yield, so a continuation r_0 advances the dot past position i_b and creates the child summary $\nu_{c_{i_b}}$. After r_0, $\beta(\nu, b) = \mathsf{Resolved}(\nu_{c_{i_b}})$ (Definition 6), and the induction hypothesis applied at $\nu_{c_{i_b}}$ gives r_1 closing the premise. Take $r = r_0 \cdot r_1$.

Case $\beta(\nu, b) = \mathsf{Resolved}(\nu_c)$. Let n_c be the non-terminal of ν_c, $\gamma_c = \mathcal{T}(n_c) = (name_c, M_c, Bind_c, \Phi_c, \hat{\tau}_c)$ the typing rule attached to n_c, and $\hat{\tau}_c$ its conclusion (Typing Rules subsection above). The type evidence $\nu_c.\tau = \texttt{typeof}_\nu(b)$ is the elaboration of $\hat{\tau}_c$ under the bindings $Bind_c$ at ν_c. We dispatch on the syntactic form of $\hat{\tau}_c$:

- *Literal closed type* ($\hat{\tau}_c = \tau_0$, e.g. Int): elaboration gives $\nu_c.\tau = \mathsf{Exact}(\tau_0)$ directly. The comparison $\tau_0 = \hat{\tau}$ on the right-hand side is closed-type decidable, and Lemma 3 closes any open frontier of $\hat{\tau}$. This is the base of the recursion.
- *Context lookup* ($\hat{\tau}_c = \Gamma(x)$): Γ at ν_c is fixed at ν_c's dot, so $\nu_c.\tau$ is the closed type bound to x in Γ. Apply Lemma 3 to expose the name evidence of x, and the literal sub-case finishes the verdict.
- *Binding reference* ($\hat{\tau}_c = b'$ for some $b' \in Bind_c$): the conclusion of γ_c asks for the type of another child of ν_c. Apply the induction hypothesis at (ν_c, b') – the AST sub-tree rooted at ν_c's b'-target is strictly smaller than the one rooted at c_{i_b}. The continuation drives $\nu_c.\tau$ to $\mathsf{Exact}(\tau_0)$, reducing to the literal sub-case.
- *Type constructor over the above* (e.g. $\hat{\tau}_c = \hat{\tau}_1 \to \hat{\tau}_2$, where each $\hat{\tau}_i$ is one of the previous forms): recurse independently on each $\hat{\tau}_i$ and assemble the closed constructor on the resulting Exact pieces. Closure under arrow, union, negation, $\top$, $\bot$ is decidable on closed types by the assumption above Definition 11.

Other primitive premises. The four remaining primitive premises are immediate corollaries of Lemma 4 and monotonicity.

- *Context membership* ($x \in \Gamma$): Γ is fixed at the dot, since it depends only on the right-bound transforms of already-closed left siblings. Apply Lemma 3 to expose the name evidence of x as $\mathsf{Exact}(name)$ for any $name \in \mathrm{dom}(\Gamma)$ in the completion set.
- *Type relations* ($\hat{\tau}_1 = \hat{\tau}_2$ and $\hat{\tau}_1 \subseteq \hat{\tau}_2$): each side elaborates to a type-evidence node; apply Lemma 4 when the side is a binding reference and Lemma 3 otherwise, then compose continuations.
- *Child type ascription* ($\Gamma \vdash b : \hat{\tau}$): this is exactly the premise $\texttt{typeof}_\nu(b) = \hat{\tau}$ of Lemma 4.
- *Child-bound context ascription* ($\Gamma[x : \hat{\tau}] \vdash b : \hat{\sigma}$): the context update is independent of input extension, so the premise reduces to ascription under $\Gamma[x : \hat{\tau}]$. Apply Lemma 4.

Lemma 5 (Rule realizability). *On parser state* $\sigma(s)$ *if* $eval_{impl}(\Phi) = \mathsf{Live}$*, there exists a continuation* r *such that at* $\sigma(sr)$*,* $eval_{impl}(\Phi) = \mathsf{Satisfied}$.

Proof. The premise list Φ of γ induces a finite sub-hypergraph $\mathcal{G}_\gamma \subseteq \mathcal{G}(s)$ (Definition 7): each $\phi \in \Phi$ contributes constraint edges, each binding $b \in Bind$ contributes the binding-reference vertex $\beta(\nu, b)$ (Definition 6). We induct on the lexicographic measure $\mu(\mathcal{G}_\gamma) = (\,|F(\mathcal{G}_\gamma)|,\, |\Phi_{\mathsf{Live}}|\,)$ where F is the set of frontier vertices (Definition 8) and Φ_{Live} counts premises with verdict Live. Both are finite by Definition 7, so μ is well-founded.

Base case ($\mu = (0, 0)$). Every binding-reference is Resolved, every typed evidence vertex is Exact, every premise is Satisfied. Take $r = \varepsilon$.

Inductive step. Pick a frontier vertex v of $\mathcal{G}_\gamma$. By the trichotomy of Definition 8, v is either (i) a Term leaf with status in $\{\mathsf{Prefix}, \mathsf{Extensible}\}$, dispatched to Lemma 3; (ii) a binding-reference $\beta(\nu, b) = \mathsf{Pending}(i_b)$, where productivity (Assumption 1) supplies a continuation that advances the dot past position i_b and so flips the verdict to $\mathsf{Resolved}(\nu_{c_{i_b}})$; or (iii) an internal summary ν with $\rho(\nu) \in \{\mathsf{Prefix}, \mathsf{Extensible}\}$, which by Definition 5 unfolds to a strictly smaller sub-graph $\mathcal{G}_\nu$ on which the induction hypothesis applies. In each case the continuation r_v either drops v from F or, by the realizability of `typeof` (Lemma 4) or of a primitive type-evidence node (Lemma 3), turns one premise from Live into Satisfied. By Remark 1, a Lost judgement on types can never be recovered to Live, so either way μ strictly decreases. Composing the r_v in dot order yields the required r.

Context Transforms

A typing rule may export a *right-bound context transform* $\nabla : \Gamma \mapsto \Gamma[x : \hat{\tau}]$ in its conclusion, written

$$\frac{\phi_1 \quad \cdots \quad \phi_n}{\Gamma \to \Gamma[x : \hat{\tau}] \vdash \hat{\sigma}} \ (name).$$

Exact nodes may pass ∇ to their right siblings via right-bound context flow (§4.3). Prefix nodes never export ∇ since their right boundary is not yet fixed. Transforms compose left to right: $\nabla_3 = \nabla_2 \circ \nabla_1$.

Theorem 1 (Typing implementation computes the ideal evaluator). *For every reachable state $\sigma(s)$,*

$$\mathsf{eval}_{\mathrm{impl}}(\mathcal{G}(s)) \ = \ \mathsf{eval}(\mathcal{G}(s)).$$

where $\mathsf{eval}(\mathcal{G}(s))$ is the ideal evaluator defined in 2.3.

Proof. We induct on the AST t of $\sigma(s)$ and prove the stronger statement that $\mathit{eval}_{\mathrm{impl}}(\nu) = \mathit{eval}(\mathcal{G}_\nu(s))$ for every node ν of t, where $\mathcal{G}_\nu(s)$ is the sub-evidence-graph rooted at ν. The theorem is the case where ν is the root of t.

Terminal leaf. A leaf $\mathsf{Term}(x, \xi, r)$ contributes one vertex and no constraint edges. By Definition 8 the singleton sub-graph is Closed iff its only vertex is non-frontier, i.e. $\xi = \mathsf{Exact}$. By Lemma 3 every Prefix or Extensible leaf has a witness, so $\mathit{eval}(\mathcal{G}_\nu(s)) = \mathsf{Live}$ for $\xi \in \{\mathsf{Prefix}, \mathsf{Extensible}\}$. This matches the terminal clause of $\mathit{eval}_{\mathrm{impl}}$.

Internal node. Let $\nu = \langle n, p_a^n, \chi \rangle$, write $V_n = \widehat{\mathcal{T}}(n)$, and write $V_c = \mathit{eval}_{\mathrm{impl}}(c)$ for $c \in \chi$. By the induction hypothesis, every V_c matches $\mathit{eval}(\mathcal{G}_c(s))$. The sub-graph $\mathcal{G}_\nu(s)$ decomposes as the union of the children's sub-graphs and the constraint edges contributed by $\mathcal{T}(n)$'s premises (Definition 7). We dispatch on the join $V_n \sqcup \bigsqcup_c V_c$:

- *Join is* Lost: either $V_n = \mathsf{Lost}$ (a rule premise is contradicted at ν) or some $V_c = \mathsf{Lost}$ (some child sub-graph contains a contradiction). In either case $\mathcal{G}_\nu(s)$ contains a contradiction in the constraint domain, and by the Lost clause of Definition 11 we have $\mathit{eval}(\mathcal{G}_\nu(s)) = \mathsf{Lost}$.

- *Join is* Satisfied: $V_n = \mathsf{Satisfied}$ and every $V_c = \mathsf{Satisfied}$. By induction every child sub-graph is Closed and free of contradiction, and by definition of $\mathit{eval}_{\mathrm{impl}}(\mathcal{T}(n))$ the rule premises hold. Hence $\mathcal{G}_\nu(s) \in \mathsf{Closed}$ and $\mathit{eval}(\mathcal{G}_\nu(s)) = \mathsf{Satisfied}$.
- *Join is* Live: at least one of V_n, V_c is Live and none is Lost. By the syntactic-state propagation rules of §2.2, only the rightmost descendant chain may be non-Exact, so at most one child c^* has $V_{c^*} = \mathsf{Live}$. The induction hypothesis at c^* supplies a continuation r_c closing $\mathcal{G}_{c^*}(s)$ into Satisfied. After r_c the rule verdict at ν is non-Lost by Remark 1, so Lemma 5 gives a further continuation r_n closing the rule premises. The composite $r = r_c \cdot r_n$ is a witness for $\mathcal{G}_\nu(s)$, so $\mathit{eval}(\mathcal{G}_\nu(s)) = \mathsf{Live}$.

The verdicts agree at ν, completing the induction.

3.1 Simply Typed Lambda Calculus Example

As a reference point, we implement the simply-typed lambda calculus (STLC) using our type system and grammar definition format.

Syntax The STLC syntax can be expressed in BNF-like notation with rule annotations and bindings:

$$\begin{aligned}
\text{Identifier} &\to \mathcal{R}(\texttt{[A-Za-z_][A-Za-z0-9]}^\star) \\
\text{Variable} &\to \text{Identifier}[x] && (\textsc{var}) \\
\text{Type} &\to \text{Identifier} \mid \text{'('} \text{ Type } \text{')'} \mid \text{Type } \text{'}\to\text{'} \text{ Type} \\
\text{Lambda} &\to \text{'}\lambda\text{'} \text{ Identifier}[a] \text{ ':'} \text{ Type}[\tau] \text{ '.'} \text{ Expression}[e] && (\textsc{lambda}) \\
\text{Application} &\to \text{Expression}[l] \text{ Expression}[r] && (\textsc{app}) \\
\text{Expression} &\to \text{Variable} \mid \text{Lambda} \mid \text{Application} \mid \text{'('} \text{ Expression } \text{')'}
\end{aligned}$$

Lambda rule.

$$\frac{\Gamma[a{:}\tau] \vdash e : B}{\tau \to B}\ (\textsc{lambda})$$

Here $\mathcal{M} = \{B\}$, $\mathcal{B} = \{a, \tau, e\}$, the single premise extends the context with the annotated binder, and the conclusion is the arrow type.

Application rule.

$$\frac{\Gamma \vdash l : A \to B \quad \Gamma \vdash r : A}{B}\ (\textsc{app})$$

A is unified between the two premises: whatever function type l resolves to determines the required type of r.

Variable rule.

$$\frac{x \in \Gamma}{\Gamma(x)}\ (\textsc{var})$$

Here $\mathcal{M} = \emptyset$, $\mathcal{B} = \{x\}$, the premise is a typing judgment for a variable, and the conclusion is its type.

Partial examples

Valid If we feed the prefix $\lambda x : \text{Int}$, the parser produces a prefix node for the lambda production. The type child is on the right frontier, so the lambda premise is unknown but not contradictory. The oracle accepts a token that extends the type, for example $\to \text{Bool}$, and it also accepts a . token that closes the annotation and starts the body.

Invalid If we feed the prefix $\lambda f : \text{Int} \to \text{Bool}.\ f\ (\lambda y : \text{Int}.\ y\ ($, the final parenthesis leaves the term syntactically open, but the semantic contradiction is already stable. The variable f has type $\text{Int} \to \text{Bool}$, so its argument must have type Int. The argument has already committed to the lambda alternative, so any successful completion of that subtree would have arrow shape $\text{Int} \to B$ for some body type B. No future token can turn a lambda into an integer argument, so the oracle rejects the prefix without waiting for the closing parenthesis.

4 Prefix Parsing Algorithm

The core algorithm is a language-parametric prefix parser. While we developed it for constrained generation, it can also serve as a static analysis tool, a completion engine, or a full type checker.

4.1 Language-Parametric Interface

The framework is parametric over SPGs, materialized in `.auf` specifications: a file defines productions, binding annotations, and semantic rules. The engine checks well-formedness, builds binding maps as in Section 2.2, and then exposes the same prefix oracle for every language instance.

Obligations

The typing rules defined in Section 3 are not resolved over finished trees. Instead, they are computed at parse time and used for branch pruning through an *obligation* system, which is the parser-time realization of the binding-resolution map β of Definition 6. The behavior is structurally equivalent, but divides the evaluation step into two operations : (i) obligation instantiation and (ii) final resolution. Pruning is done thanks to the binding information gathered in (i).

Definition 12 (Obligation and store). *An* obligation store Ω *records the evidence expected from bound children. Each obligation has the form*

$$\omega_b \ = \ (b,\ v_b,\ E_b),$$

where b is the binding name, v_b is terminal or span evidence, and E_b is type evidence. An obligation ω_b with both $v_b = \bot$ and $E_b = \bot$ is the parser-side encoding of $\beta(\nu, b) = \mathsf{Pending}(i_b)$; resolving a terminal child stores terminal evidence and yields $\beta(\nu, b) = \mathsf{Resolved}(\mathsf{Term}(\cdot))$; resolving a non-terminal child stores span and exported type evidence and yields $\beta(\nu, b) = \mathsf{Resolved}(\nu_{c_{i_b}})$.

The purpose of obligations is to provide an interface for rule evaluation that behaves like recursive descent, while integrating with the parser to reduce complexity and enable pruning. Equivalently, the store materializes the Pending-to-Resolved transitions of β along the dot.

4.2 Parsing

A *prefix parser* for an SPG G is a function

$$\Psi_G : \Sigma^* \to \mathcal{F}_\partial \cup \{\texttt{reject}\}.$$

It is defined by a runtime state γ that includes:

- The input segmentation $x = \mathsf{seg}_{\mathcal{L}}(s) = (x_0, \ldots, x_{m-1})$
- The agenda $\mathcal{A}$ of items to process
- The chart C mapping recognized nonterminal spans to node identifiers
- The waiter relation W mapping nonterminal spans to paused parent items
- The set R of known end positions for each nonterminal and segment index
- The end-of-input frontier F of items awaiting EOF closure
- The packed parse arena H

A live item is

$$\eta = \langle n, p, k, [i, j), \Gamma, \Omega, \chi \rangle$$

where n is the non-terminal being recognized, $p = p_a^n$ is one of its productions, k is the dot position, $[i, j)$ is the consumed segment span, Γ is the semantic context at the dot, Ω is the current obligation store, and χ is the ordered list of accepted child references and terminal evidence.

$$p = \underbrace{\alpha_1[b_1] \ \cdots \ \alpha_k[b_k]}_{\text{recognized}} \bullet \underbrace{\alpha_{k+1}[b_{k+1}] \ \cdots \ \alpha_n[b_n]}_{\text{remaining}}$$

Each arena node is the evidence summary ν of Definition 5, viewed as a packed handle for a family of parses and extended with a resolved type τ that makes already evaluated semantic resolution available and a context transform ∇ that exposes how the node will modify right-bound context.

$$\nu = \langle n, [i, j), \rho, \tau, \nabla, \beta \rangle$$

with $\rho \in \{\mathsf{Exact}, \mathsf{Prefix}, \mathsf{Extensible}\}$, τ the exported semantic type information of the typing-domain instance (§3), ∇ an optional right-bound context transform, and β the parent-visible binding-resolution map (Definition 6), where $\beta(\nu, b)$ is realized by the obligation $\omega_b \in \Omega$ of Definition 12. Exact nodes are closed on the current input and may export ∇. Prefix nodes are syntactically unfinished at the right frontier. Extensible nodes are fully recognized on the current input but contain right-frontier terminal evidence that may still grow. Prefix and Extensible nodes never export context transforms.

Semantic Interface The parser uses the following operations. The names match the implementation-level operations; mathematically, mutating operations are written as functional updates. They are

partial: failure means that the branch is discarded.

$$\begin{aligned}
\mathsf{create}(G, p, r) &: \text{creates production-local obligations for production } p \text{ at root } r,\\
\mathsf{step}(\Omega, k, a) &: \text{projects obligations through child position } k \text{ of alternative } a,\\
\mathsf{prune}(\Omega, n, G) &: \text{returns the alternatives of } n \text{ compatible with } \Omega,\\
\mathsf{for}(\Omega, a) &: \text{keeps only obligations compatible with seeded alternative } a,\\
\mathsf{at}(\Omega, k) &: \text{re-roots obligations at child position } k,\\
\mathsf{descend}(p, b, \Gamma, \Omega) &: \text{computes the context used to enter child binding } b,\\
\mathsf{resolve}_T(\Omega, k, a, \mathsf{Term}(x, \zeta, r)) &: \text{stores terminal evidence for obligations at } (k, a),\\
\mathsf{resolve}_N(\Omega, k, a, \nu) &: \text{stores span and type evidence from accepted child node } \nu,\\
\mathsf{finalize}(p, \Gamma, \Omega, \rho) &: \text{evaluates the production rule when finishing a node},\\
\mathsf{apply}(\Gamma, \nabla) &: \text{applies an exported right-bound context transform.}
\end{aligned}$$

We assume all semantic operations are denotational filters: defined outputs denote exactly the satisfying refinements, and semantic rejection happens only on empty denotation.

Push Transitions All parser table updates are written as

$$\gamma \downarrow_{\alpha} (y),$$

where α names the table or helper being updated and y is the payload supplied to that update. The update labels are

$$\alpha ::= \mathsf{process} \mid \mathsf{waiters} \mid \mathsf{frontier} \mid \mathsf{node} \mid \mathsf{record} \mid \mathsf{complete}.$$

The intended payloads are:

$$\begin{aligned}
\gamma \downarrow_{\mathsf{process}} (\eta) &: \text{interns a process task in the agenda, subject to the item key},\\
\gamma \downarrow_{\mathsf{waiters}} (n, j, \eta) &: \text{interns the paused parent item } \eta \text{ in } W(n, j),\\
\gamma \downarrow_{\mathsf{frontier}} (\eta) &: \text{appends } \eta \text{ to } F,\\
\gamma \downarrow_{\mathsf{node}} (\nu, \pi) &: \text{interns a node and packed alternative in } H,\\
\gamma \downarrow_{\mathsf{record}} (c) &: \text{records a completed node in } C \text{ and its end position in } R,\\
\gamma \downarrow_{\mathsf{complete}} (c) &: \text{appends a completion task and wakes matching waiters.}
\end{aligned}$$

When a push allocates a fresh arena handle, we write the handle above the push:

$$\gamma \downarrow^{h}_{\mathsf{node}} (\nu, \pi).$$

Parse Inference The following rules summarize the semantic fields added to the standard prefix-Earley transition system. They use the push transition explicitly; standard side conditions such as deduplication and completed-dot checks are the side conditions of the corresponding push action.

We write $\Omega_1 \oplus \Omega_2$ for extending one obligation store with another.

$$\frac{q = p^n_a \in Q \subseteq P \quad \Omega^{\mathsf{in}}_q = \mathsf{for}(\Omega, a) \quad \Omega_q = \mathsf{create}(G, q, \mathsf{root}(\Omega^{\mathsf{in}}_q)) \oplus \Omega^{\mathsf{in}}_q}{\gamma \downarrow_{\mathsf{process}} (\langle n, q, 0, [j, j), \Gamma, \Omega_q, \epsilon \rangle)} \quad \textsc{Seed}$$

Seed is parameterized by a set of candidate productions Q, an input position j, an incoming context Γ, and an incoming obligation store Ω. The initial call is $Q = A(S)$, $j = 0$, $\Gamma = \Gamma_0$, and $\Omega = \mathsf{empty}$.

$$\frac{\begin{array}{c}\eta = \langle n, p_a^n, k, [i,j), \Gamma, \Omega, \chi\rangle \quad p_a^n[k] = n'[b] \quad n' \in N \\ \Gamma_c = \mathsf{descend}(p_a^n, b, \Gamma, \Omega) \quad \Omega^\dagger = \mathsf{step}(\Omega, k, a) \quad \Omega_c = \mathsf{at}(\Omega^\dagger, k)\end{array}}{\gamma \downarrow_{\mathsf{waiters}} (n', j, \eta) \qquad \forall q \in \mathsf{prune}(\Omega^\dagger, n', G), \quad \gamma \downarrow_{\mathsf{process}} (\eta_q)} \quad \textsc{ProcessNonterminal}$$

where each η_q is the child item produced by Seed with $P = \{q\}$, position j, context Γ_c, and parent obligations Ω_c. Thus pruning is applied before any child production is seeded, and $W(n', j)$ stores the paused parent item rather than the predicted child.

$$\frac{\begin{array}{c}\eta = \langle n, p_a^n, k, [i,j), \Gamma, \Omega, \chi\rangle \quad p_a^n[k] = t[b] \quad t \in T \quad \mathsf{match}(t, x_j) = (\zeta, r) \\ \zeta \in \{\mathsf{Exact}, \mathsf{Extensible}\} \quad T_j = \mathsf{Term}(x_j, \zeta, r) \quad \Omega' = \mathsf{resolve}_T(\Omega, k, a, T_j)\end{array}}{\gamma \downarrow_{\mathsf{process}} (\langle n, p_a^n, k+1, [i, j+1), \Gamma, \Omega', \chi \cdot T_j\rangle)} \quad \textsc{ConsumeReady}$$

$$\frac{\begin{array}{c}\eta = \langle n, p_a^n, k, [i,j), \Gamma, \Omega, \chi\rangle \quad p_a^n[k] = t[b] \quad t \in T \quad j = m-1 \\ \mathsf{match}(t, x_j) = (\mathsf{Prefix}, r) \quad T_j = \mathsf{Term}(x_j, \mathsf{Prefix}, r) \quad \Omega' = \mathsf{resolve}_T(\Omega, k, a, T_j)\end{array}}{\gamma \downarrow_{\mathsf{frontier}} (\langle n, p_a^n, k+1, [i, m), \Gamma, \Omega', \chi \cdot T_j\rangle)} \quad \textsc{ConsumePrefix}$$

$$\frac{\eta = \langle n, p, k, [i,m), \Gamma, \Omega, \chi\rangle \quad p[k] \in T}{\gamma \downarrow_{\mathsf{frontier}} (\eta)} \quad \textsc{ConsumeEof}$$

Let $\rho(\eta)$ be the status selected by finish:

$$\rho(\eta) = \begin{cases} \mathsf{Exact} & \text{all children are complete and none is open,} \\ \mathsf{Extensible} & \text{all children are complete and at least one is open,} \\ \mathsf{Prefix} & \text{otherwise.} \end{cases}$$

Prefix and Extensible statuses force the exported transform to $\perp$.

$$\frac{\begin{array}{c}\eta = \langle n, p, k, [i,j), \Gamma, \Omega, \chi\rangle \quad \mathsf{finishable}(\eta) \quad \rho = \rho(\eta) \quad \mathsf{finalize}(p, \Gamma, \Omega, \rho) = (\tau, \nabla, \spadesuit) \\ \nabla_\rho = \begin{cases} \nabla & \rho = \mathsf{Exact}, \\ \perp & \rho \in \{\mathsf{Prefix}, \mathsf{Extensible}\}, \end{cases} \qquad \nu = \langle n, [i,j), \rho, \tau, \nabla_\rho, \mathsf{bindings}(\Omega)\rangle \quad \pi = (p, \chi)\end{array}}{\gamma \downarrow^h_{\mathsf{node}} (\nu, \pi) \qquad \gamma \downarrow_{\mathsf{complete}} (\langle n, i, j, h\rangle)} \quad \textsc{Finish}$$

Here $\mathsf{finishable}(\eta)$ means either that the main process loop has reached the end of the production, or that η has been identified as partial from the EOF frontier. The value $\spadesuit$ is the implementation's semantic-completeness flag status, corresponding to the *eval* 11 return values. It is used to trigger branch abandon if it is Lost. Finalize acts as a sort of "closing" for a *eval* call.

$$\frac{\begin{array}{c}c = \langle n', j, \ell, h\rangle \quad \eta = \langle n, p_a^n, k, [i,j), \Gamma, \Omega, \chi\rangle \in W(n', j) \quad H[h] = \nu = \langle n', [j, \ell), \rho, \tau, \nabla, \beta\rangle \\ \Omega' = \mathsf{resolve}_N(\Omega, k, a, \nu) \quad \Gamma' = \begin{cases} \mathsf{apply}(\Gamma, \nabla) & \nabla \neq \perp, \\ \Gamma & \nabla = \perp, \end{cases} \quad \eta' = \mathsf{resume}(\eta, h)\end{array}}{\gamma \downarrow_{\mathsf{process}} (\eta')} \quad \textsc{CompleteResume}$$

The completion c is pushed once, by FINISH, with

$$\gamma \downarrow_{\mathsf{complete}} (c).$$

Then the resume step above is repeated for each waiter in $W(n', j)$ for which resume is defined.

After the main agenda is saturated, we do *frontier lifting* on F by repeatedly applying FINISH and COMPLETERESUME to frontier items and their resumed parents. It does not call process, seed, or consume, but rather it only finishes frontier summaries and propagates them through the existing waiter network. This is a central element to what makes our parser able to process *prefixes*

The parser succeeds when saturation and EOF closure produce a root node

$$\nu = \langle S, [0, m), \rho, \tau, \nabla, \beta \rangle \quad \rho \in \{\mathsf{Exact}, \mathsf{Prefix}, \mathsf{Extensible}\}.$$

If no such root node is produced, the parser returns `reject`.

4.3 Context Flow

The context stored in an item is the context at the dot. It is obtained by combining two forms of context flow: top-down context passed into a child, and right-bound context exported by exact children to their right siblings.

Downward context When the parser predicts a child $p[k] = n'[b]$, it computes the child context by

$$\Gamma_{\mathsf{child}} = \mathsf{descend}(p, b, \Gamma, \Omega).$$

This is a downward flow: the child is parsed under the context selected by the parent rule. For example, the body of a lambda is predicted under $\Gamma[x : \tau]$, but this extension is local to that child. This property is inherited from recursive descent, where it is more apparent.

Right-bound flow Right-bound flow happens after a child is completed. If exact children to the left of the dot export transforms

$$\nabla_1, \ldots, \nabla_q,$$

then the context at the dot is

$$\Gamma_q = (\nabla_q \circ \cdots \circ \nabla_1)(\Gamma_0).$$

Accepting the next exact child with export ∇_{q+1} updates the parent context to

$$\Gamma_{q+1} = \nabla_{q+1}(\Gamma_q) = (\nabla_{q+1} \circ \nabla_q \circ \cdots \circ \nabla_1)(\Gamma_0).$$

Non-exact nodes do not participate in this composition: Prefix and Extensible nodes export $\bot$, so resuming such a child leaves the parent context unchanged. Thus top-down flow determines the context used inside a child, while right-bound flow determines the context seen by later siblings.

STLC example trace In B we demonstrate the parser transition behavior with a simple STLC example.

4.4 Soundness

The parser is used as a prefix oracle. The property we need is that every accepted prefix can still be completed into a closed semantic parse.

The invariant is simple: accepted means not dead. Safe pruning lets the parser work over an infinite continuation tree because it only cuts branches that cannot hold in the semantic domain. We use two invariants of the rules above. First, after erasing semantic fields, the rules form the standard Earley prefix parser for the syntactic projection of G. Second, semantic operations are exact local filters: a semantic operation is defined exactly when the evidence graph carried by the current item or node is not lost. In particular, exact finalization requires a closed satisfying graph, while non-exact finalization requires a live graph in the sense of Definition 11.

Theorem 2 (Completability soundness). *Let $G \in SPG(D)$ be productive by Assumption 1. If*

$$\Psi_G(s) \neq \mathsf{reject},$$

then there exists $s' \in \Sigma^*$ such that $\Psi_G(ss')$ produces an Exact *root node.*

Proof. Since $\Psi_G(s) \neq \mathsf{reject}$, saturation on s produced a root node

$$\nu = \langle S, [0, m), \rho, \tau, \nabla, \beta \rangle$$

with $\rho \in \{\mathsf{Exact}, \mathsf{Prefix}, \mathsf{Extensible}\}$.

If $\rho = \mathsf{Exact}$, the root is already closed on the current input, so taking $s' = \varepsilon$ proves the result.

Suppose instead that $\rho \in \{\mathsf{Prefix}, \mathsf{Extensible}\}$. This root was produced by non-exact finalization. By the semantic side condition of FINISH, the ♠ flag of $eval_{\mathrm{impl}}$ on ν is non-Lost, and by Theorem 1 this matches $eval(\mathcal{G}(s))$ on the same evidence summary. Since $\rho \neq \mathsf{Exact}$, at least one frontier vertex remains in the sense of Definition 8, so $eval(\mathcal{G}(s)) = \mathsf{Live}$. By Definition 11, this means

$$\exists s' \in \Sigma^*.\ \mathcal{G}(s \cdot s') \in \mathsf{Closed},$$

and by Definition 10, such an s' is precisely a witness $s' \in \mathcal{D}(\mathcal{G}(s))$.

It remains only to justify that the parser will actually produce the closed root on ss'. The suffix s' closes the branch summarized by ν. Syntactically, the erased inference system is the Earley prefix parser, so the corresponding syntactic continuation is admitted. Semantically, each operation on that branch is an exact filter, so following a satisfying continuation cannot make a required semantic operation fail. Since the system is saturated, all items and nodes on this closed branch are eventually inserted. In particular, $\Psi_G(ss')$ produces an Exact root node.

Lemma 6 (Prefix completeness of saturation). *Let $G \in \mathsf{SPG}(D)$. If $\Psi_G(sr)$ produces an* Exact *root node, then*

$$\Psi_G(s) \neq \mathsf{reject}.$$

Proof. Take a saturated derivation of the exact root over sr, and cut it at the input frontier after s. By prefix-awareness of the segmentation policy, closed interior segments are unchanged by this cut; only the final segment may become open.

The syntactic projection of the cut derivation is therefore a valid prefix Earley derivation for s. Semantic evidence already to the left of the cut is unchanged. Evidence crossing the cut becomes prefix evidence, and it is retained because the complete derivation over sr gives an actual suffix closing it.

Moreover, semantic graphs grow monotonically with input:

$$\mathcal{E}(s) \subseteq \mathcal{E}(sr) \qquad \text{and} \qquad \mathcal{C}(s) \subseteq \mathcal{C}(sr).$$

This holds because each parser transition only adds vertices and edges: Seed and ProcessNonterminal insert fresh items, Finish interns a node ν, and CompleteResume flips $\beta(\nu, b)$ from $\mathsf{Pending}(i_b)$ to $\mathsf{Resolved}(\nu_{c_{i_b}})$ (Definition 6) without removing the binding-reference vertex. The exact root over sr has a closed satisfying graph. Restricting that closed graph to the evidence and constraints already present at s gives a satisfying grounding for the prefix graph. Hence none of the semantic operations in the cut derivation is lost. Since the inference system is saturated, the corresponding exact or non-exact root node is inserted when parsing s. Therefore $\Psi_G(s) \neq \mathsf{reject}$.

Theorem 3 (Prefix monotonicity). *Let $G \in \mathsf{SPG}(D)$ be productive by 1. If*

$$\Psi_G(s) \neq \mathsf{reject},$$

then for every prefix s'' of s,

$$\Psi_G(s'') \neq \mathsf{reject}.$$

Proof. Let $s = s''r$. By Theorem 2, there exists $t \in \Sigma^*$ such that

$$\Psi_G(st) = \Psi_G(s''rt)$$

produces an Exact root node. Applying Lemma 6 to the complete input $s''rt$ gives

$$\Psi_G(s'') \neq \mathsf{reject}.$$

5 P7 Constrained Generation

P7 is the model-facing layer that uses the Aufbau prefix oracle during decoding. The shared core does not depend on any particular language model: the LLM proposes continuations, and the semantic prefix oracle removes continuations that would leave the live syntactic and semantic prefix space.

5.1 System Architecture

Figure 2 shows the overall system architecture and generation loop. Unlike logit-filtering approaches, P7 does not ask Aufbau for a full admissible vocabulary before sampling. Instead, sampling happens on the GPU. The sampled token is then checked by Aufbau using `try_feed`. If the token is rejected, P7 masks that token for the current step and resamples. This approximates sampling from a distribution containing allowed elements only but has better average complexity, since in most cases the good token will be among the highest if the model has some language data.

5.2 Constrained Decoding

The constrained decoding system, implemented in P7, integrates Aufbau with Hugging Face Transformers [21]. The generation loop is illustrated in Figure 2.

At each decoding step:

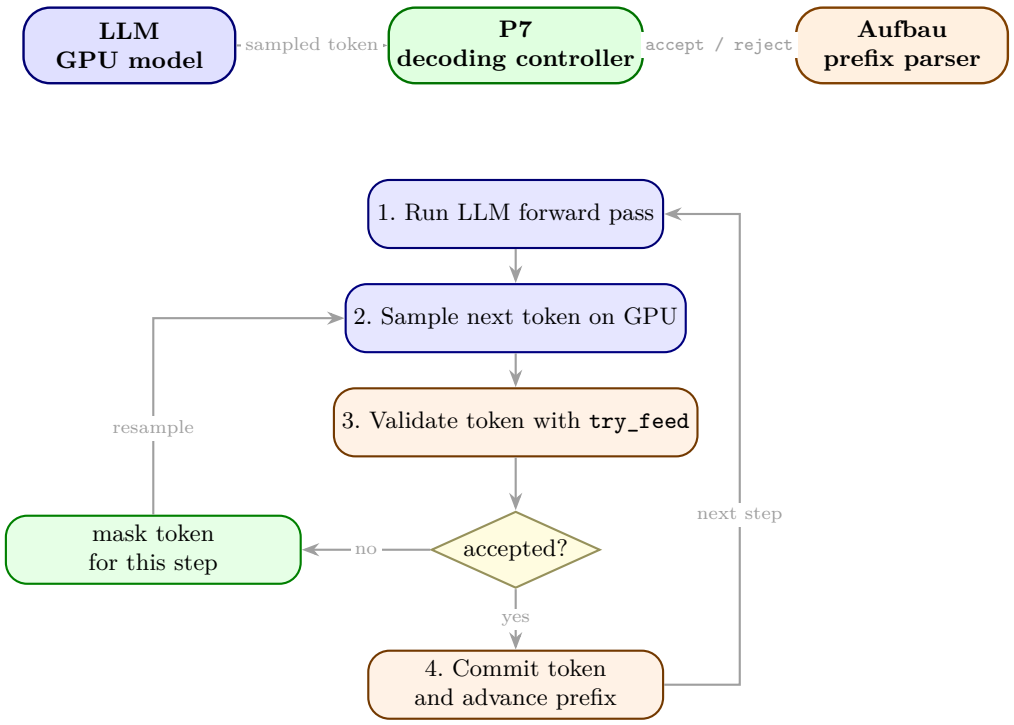

Fig. 2. P7 system structure and constrained decoding loop. The language model proposes tokens, P7 validates them with Aufbau through `try_feed`, rejected tokens are masked for the current step, and accepted tokens are committed to the prefix.

1. The LLM computes logits for the next token.
2. A token is sampled on the GPU from the current distribution.
3. P7 passes the sampled token to AUFBAU using `try_feed`.
4. If AUFBAU accepts the token, P7 commits it and advances the prefix.
5. If AUFBAU rejects the token, P7 masks that token for the current step and resamples.

Thus P7 uses AUFBAU as an online validation oracle rather than as a precomputed vocabulary filter. This avoids constructing the full admissible token set at every step. The decoder only queries the semantic parser for tokens that the model actually samples. Deterministic decoding can be implemented by sampling from a degenerate distribution or by repeatedly selecting the highest unmasked token and checking it with `try_feed`. [1]

Token-level interface and termination The formal parser is defined over character strings, while a model emits tokens from a finite tokenizer vocabulary V. At a decoding state with committed text s, a token $a \in V$ is interpreted by its decoded text fragment $\mathtt{dec}(a)$. The call $\mathtt{try\_feed}(a)$

[1] While not the most efficient, and maybe too probabilistic for some applications, this approach avoids copying back and forth between CPU and GPU for the full vocabulary.

accepts exactly when

$$\Psi_G(s\,\mathsf{dec}(a)) \neq \texttt{reject}.$$

Thus token validation is reduced to the same character-level prefix oracle used in the formal semantics. The tokenizer layer does not need to precompute the full admissible vocabulary; it only asks whether the sampled token's decoded string keeps the current text in the semantic prefix language. The regular-terminal completability machinery used for this bridge follows the prefix-completion construction described in [7].

The resampling loop terminates because V is finite. During one decoding step, P7 maintains a masked set $M \subseteq V$. A rejected token is inserted into M and cannot be sampled again at that step. The loop stops either when some token is accepted and committed, or when $M = V$. In the latter case all tokens, including any end-of-sequence token present in V, have been rejected for the current prefix, so generation reports that the state is blocked rather than continuing to resample.

5.3 Proof-Search Graph Perspective

The decoding process can be read as graph search over semantic parser states. Nodes are parser configurations together with outstanding semantic obligations; edges are token or tree-expansion choices; accepting an edge means the target state still has a semantic prefix parse. The language model supplies a probability distribution over outgoing edges, while AUFBAU removes edges whose targets contain stable syntactic or semantic contradiction. This does not prove that the model selects the best proof or program, but it ensures that the generated trace remains inside the semantically live portion of the search graph.

6 Evaluation

The evaluation separates oracle validation from model-facing generation. The first part checks that the implementation follows the semantic prefix model; the second part measures what the oracle changes when placed in the P7 decoding loop.

6.1 Oracle Validation

In order to evaluate the correctness of the AUFBAU engine during development, we built a validation suite that tests the completability property from Theorem 2. We take a string $s = c_0 \cdots c_n,\ c \in \Sigma$, and we ensure that every prefix $c_0 \cdots c_k,\ k \leq n$ is accepted by the parser. We also verify *completion monotonicity* by running an AUFBAU backed program synthesis job on each prefix, and ensuring the number of found complete completions decreases as k increases.

6.2 Constrained vs. Unconstrained Generation

In Figure 6, task-invalid outputs are syntactically accepted programs that fail the benchmark resolver, incomplete outputs remain unfinished at the token budget, and non-completable outputs cannot be extended to a valid formal program.

To evaluate the real-world performance benefits of constrained generation in specific program synthesis tasks, we compare across open models up to 31 billion parameters the success rate on a custom benchmark mixing simple instruction-following, coding tasks and logic problems. Our

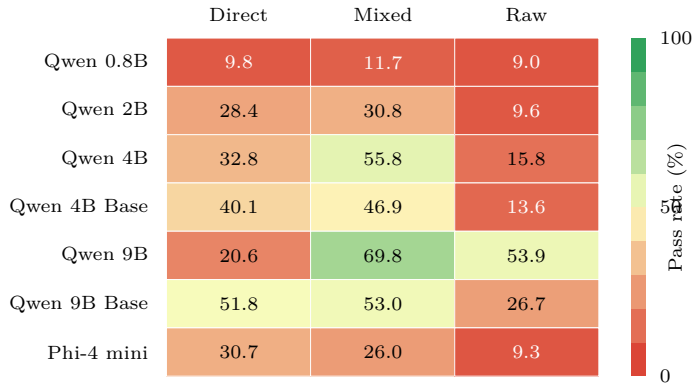

Fig. 3. Matrix heatmap showing constraint satisfaction across different model sizes and grammar types.

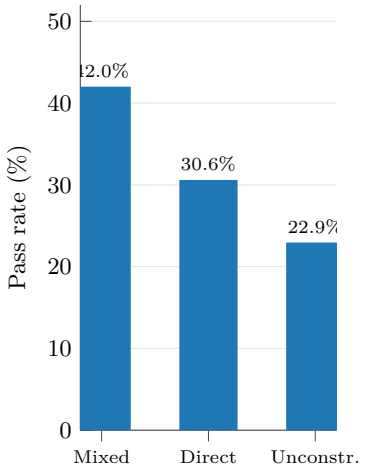

Fig. 4. Pass rate comparison across constrained and unconstrained generation

benchmark tasks use the `STLC`, `IMP` and `FUN` languages. In order to validate, we first ensure the program is complete and well-typed, then, using a small interpreter we implemented in Python, and three task resolution models we validated for task-validity:

- We check for program equivalence between the LLM produced result and an expected code string, normalizing with β-reduction and α-reduction in the FUN and STLC cases. We check for type and structural correspondence.
- We match the output against a series of task-defined tests that it must pass to be considered correct. In the case of factorial, the resolution was computing factorials up to n. With some Kolmogorov complexity approximation we know that in the given token restriction the LLM could not have possibly cheated.
- We set an expected context state, and match it up against the produced context after executing the LLM-produced code.

In the suite, we have 3 decoding modes:

1. **Unconstrained (raw)** meaning the model must generate directly the expected output in a N token limit
2. **Constrained (direct)** where the models must generate directly the expected output, but under P7 constraints, in the same N token limit
3. **Constrained (mixed)** where we mix an unconstrained generation stream for M tokens, and a formally constrained output block for N tokens.

All the models have the exact same prompt, containing grammar details, examples, and clear instructions about the tasks. As a reference point big models like GPT-5.3 managed to pass 100% of the tests.

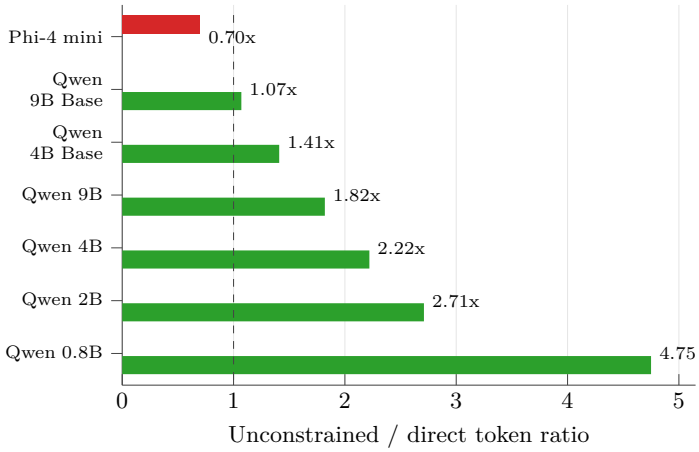

Fig. 5. Token efficiency: unconstrained vs. constrained direct generation

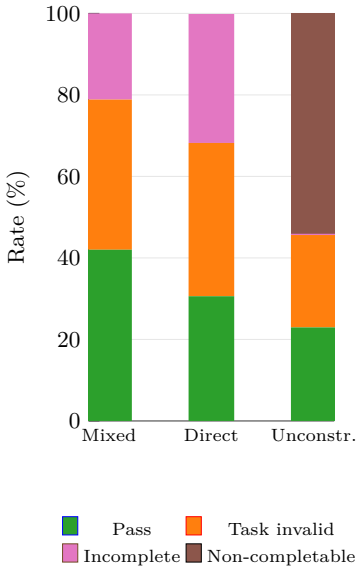

Fig. 6. Error repartition by mode

Table 1. Representative examples of benchmark tasks

ID	Prompt	Initial Prefix
`stlc_apply_twice_int`	Apply function twice to Int	`λ f:(Int->Int).`
`imp_while_factorial_four`	Compute factorial using while loop	`{ let n: Int =`
`fun_compose`	Compose two functions	`let compose:`

Leaderboard To see how constrained generation helps smaller models fare against bigger ones, we ran the unconstrained workflow on closed-source and bigger models, using the OpenRouter API (GPT-5.4-mini, gemma-4-31b-it). As a result, we see an impressive performance of smaller open models under P7, with Qwen-3.6-27B surpassing GPT-5.4-mini.

Limitations and real-world targets While the results are impressive, we can't assume they fully port to general programming tasks, as we use specific languages such as FUN, IMP, or even STLC which is probably underrepresented in coder models.

However, it is much closer to tool-use in agentic workflows, or API interaction. Indeed, in those settings, the model has to interact with conventions and languages that are possibly out of the training data, and might include context-dependent constraints.

This is why such environments are a better target for our system. Contrary to other work in this space (see Section 7), this system is not made for long program generation tasks, but rather for varied interactions with specialized formal languages.

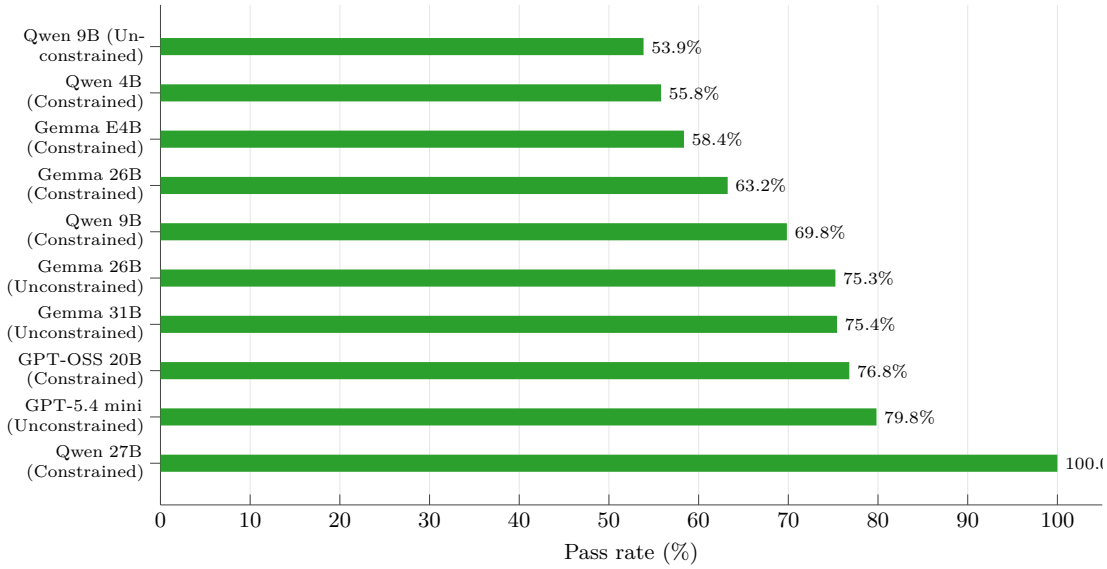

Fig. 7. Top 10 scores on benchmarks with both constrained and unconstrained models

7 Related Work

Formal language foundations. Our syntactic core is classical: Earley parsing gives a sound and complete dynamic-programming recognizer for CFGs [4], and packed forests/GLR-style sharing are standard tools for ambiguous grammars [18]. The semantic layer is grounded in attribute grammars and syntax-directed definitions [6]: productions expose child evidence, and rules compute semantic facts from that evidence. Demers, Reps, and Teitelbaum [3] give the foundational online reading: their compound dependency graph and *available*/*unavailable* attribute split is the direct ancestor of our evidence graph and of the Resolved/Pending verdicts of β. Hazel's typed-hole calculus and marked recovery [10,22] sit in the same lineage but for a structure editor rather than an online parser. Constraint and logic grammar formalisms, including definite-clause grammars [13] and unification-based grammars [16], similarly accept derivations by solving attached constraints. Our class should be read as a generalized, decidable and realizable fragment of this lineage. The difference is combining the online setting with the obligation-based pruning that allows us to resolve ambiguous grammars that might produce an infinity of trees.

Grammar-constrained decoding. PICARD [15] popularized incremental parsing as a token rejection mechanism for text-to-SQL. Outlines [20] and later grammar-constrained systems enforce regular or context-free structure during generation. DOMINO [1] and Park et al. [12] sharpen the tokenizer-alignment problem and show how much efficiency depends on aligning subword vocabularies with grammar terminals. Formatron [17] is the closest systems baseline for our parser architecture: it uses Earley-driven dynamic pruning, caching, and related engineering optimizations for efficient structured decoding. The distinction is semantic: Formatron targets syntactic structure such as regexes, CFGs, and JSON schemas, whereas our oracle carries prefix-conservative semantic obliga-

tions, binding evidence, and context transforms. These systems provide the syntactic baseline that our semantic prefix oracle generalizes.

Semantic and type constraints. Synchromesh [14] constrains code generation with syntax, scope, typing, and contextual logic in concrete domains. Mündler et al. [8] give a type-constrained decoder based on prefix automata and inhabitable type search, closest to our type-directed instance. ChopChop [9] is closest in spirit at the semantic level, but the main differences are our constraint targets: while ChopChop implements generic constraints over an output space, we offer a drop-in framework to define *languages*.

Structured output systems. JSON-schema and structured-output benchmarks [5] show that constrained decoding is becoming infrastructure, not a niche technique. Systems such as SGLang [23] expose structured generation as a programming interface. Our work targets the next layer: outputs whose validity depends on semantic state accumulated during parsing, not only on schema shape.

8 Limitations

Language coverage. The current semantic algebra supports finite typing contexts, is quite limited, and could only be extended to a small subset of real world general purpose programming languages. Realistic TypeScript or Python fragments would require additional premise forms (for instance row polymorphism, structural subtyping, and effect tracking) and broader context domains (module-level scopes, mutable cells, refinement information). We expect these extensions to fit the same parser architecture, but each new premise form has to be checked for realizability before it can be admitted into the framework. This might be hard due to second order logic requirement we cannot implement yet.

Performance. While overhead is manageable for the grammars exercised in our evaluation, large grammars with many typing rules, deep nesting, or expensive obligation resolution can slow decoding. Cost grows with the number of live items kept on the agenda, with the size of the obligation store on each item, and with the cost of evaluating individual premises. Industrial-scale grammars would benefit from caching and chart sharing strategies that are not yet implemented in Aufbau.

9 Conclusion

We presented semantic prefix grammars, a language-parametric framework for constrained generation with context-dependent semantic checks over prefix parse forests. The core idea is to keep syntax ordinary and make semantics explicit: bound productions expose evidence, semantic premises evaluate conservatively over partial input, and exact nodes alone may export context effects. This gives a principled path from CFG-only constrained decoding toward realistic programming-language fragments, even beyond the current implemented premise core. Key directions for future work are richer semantic algebras for TypeScript and Python, mechanized proofs of the semantic invariants, and tighter integration with the model level.

Acknowledgments

We thank Niels Mündler at ETH Zürich's SRI Lab for feedback on the implementation, help with references in the constrained decoding space and theoretical advice. This work was supported by Unsuspicious Industries, an independent nonprofit research collective.

References

1. Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. Guiding LLMs the right way: Fast, non-invasive constrained generation. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, pages 3658–3673, 2024.
2. Janusz A. Brzozowski. Derivatives of regular expressions. *Journal of the ACM*, 11(4):481–494, 1964.
3. Alan J. Demers, Thomas Reps, and Tim Teitelbaum. Incremental evaluation for attribute grammars with application to syntax-directed editors. In *Proceedings of the 8th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 105–116. ACM, 1981.
4. Jay Earley. An efficient context-free parsing algorithm. *Communications of the ACM*, 13(2):94–102, 1970.
5. Saibo Geng, Hudson Cooper, Michał Moskal, Samuel Jenkins, Julian Berman, Nathan Ranchin, Robert West, Eric Horvitz, and Harsha Nori. Generating structured outputs from language models: Benchmark and studies. *arXiv preprint arXiv:2501.10868*, 2025.
6. Donald E. Knuth. Semantics of context-free languages. *Mathematical Systems Theory*, 2(2):127–145, 1968.
7. Paul Kronlund-Drouault. Completeability and regular expressions. `https://unsuspicious.org/blog/completing-regex/`, 2025. Unsuspicious Industries, Oct. 12, 2025.
8. Niels Mündler, Jingxuan He, Hao Wang, Koushik Sen, Dawn Song, and Martin Vechev. Type-constrained code generation with language models. *Proceedings of the ACM on Programming Languages*, 9(PLDI), 2025.
9. Shaan Nagy, Timothy Zhou, Nadia Polikarpova, and Loris D'Antoni. Chopchop: A programmable framework for semantically constraining the output of language models. In *Proceedings of the 53rd ACM SIGPLAN Symposium on Principles of Programming Languages (POPL)*. ACM, 2026.
10. Cyrus Omar, Ian Voysey, Michael Hilton, Jonathan Aldrich, and Matthew A. Hammer. Hazelnut: A bidirectionally typed structure editor calculus. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pages 86–99. ACM, 2017.
11. Scott Owens, John Reppy, and Aaron Turon. Regular-expression derivatives reexamined. *Journal of Functional Programming*, 19(2):173–190, 2009.
12. Kanghee Park, Timothy Zhou, and Loris D'Antoni. Flexible and efficient grammar-constrained decoding. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, pages 48262–48275, 2025.
13. Fernando C. N. Pereira and David H. D. Warren. Definite clause grammars for language analysis–a survey of the formalism and a comparison with augmented transition networks. *Artificial Intelligence*, 13(3):231–278, 1980.
14. Gabriel Poesia, Oleksandr Polozov, Vu Le, Ashish Tiwari, Gustavo Soares, Christopher Meek, and Sumit Gulwani. Synchromesh: Reliable code generation from pre-trained language models. *arXiv preprint arXiv:2201.11227*, 2022.
15. Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 9895–9901. Association for Computational Linguistics, 2021.
16. Stuart M. Shieber. *An Introduction to Unification-Based Approaches to Grammar*. Number 4 in CSLI Lecture Notes. CSLI, Stanford, CA, 1986.

17. Xintong Sun, Chi Wei, Minghao Tian, and Shiwen Ni. Earley-driven dynamic pruning for efficient structured decoding. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. arXiv:2506.01151.
18. Masaru Tomita. An efficient context-free parsing algorithm for natural languages. In *Proceedings of the 9th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 455–462, 1985.
19. Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.
20. Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*, 2023.
21. Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumont, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45. Association for Computational Linguistics, 2020.
22. Eric Zhao, Justin Lubin, Yongwei Yuan, Hannah Yao, Daniel Cyrus, Jonathan Aldrich, and Cyrus Omar. Total type error localization and recovery with holes. *Proceedings of the ACM on Programming Languages*, 8(POPL), 2024.
23. Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024.

A Mixed generation results

While mixed generation has shown the best results overall, we still observe weird patterns in the benchmarks results. Most models benefited from the CoT space, but other have been disadvantaged. This might be linked to the sampling methods, or to a CoT cuttof due to a limited think token budget.

B STLC Engine Trace

We illustrate the parser on the STLC term

$$M = \lambda f : B \to C.\ \lambda g : A \to B.\ \lambda x : A.\ f\,(g\,x).$$

Using the STLC grammar conventions of Section 2, lambda binds a, τ, e, app binds l, r, and var binds x. The term has type

$$(B \to C) \to (A \to B) \to A \to C,$$

which is the Curry–Howard proof of implication transitivity:

$$(B \Rightarrow C) \Rightarrow (A \Rightarrow B) \Rightarrow A \Rightarrow C.$$

We use ConsumeReady as the transition that advances an item over a terminal child when the current segment matches that terminal as Exact or Extensible, records the resulting terminal evidence through $\mathsf{resolve}_T$, and continues the item at the next input position.

C Grammar Specifications

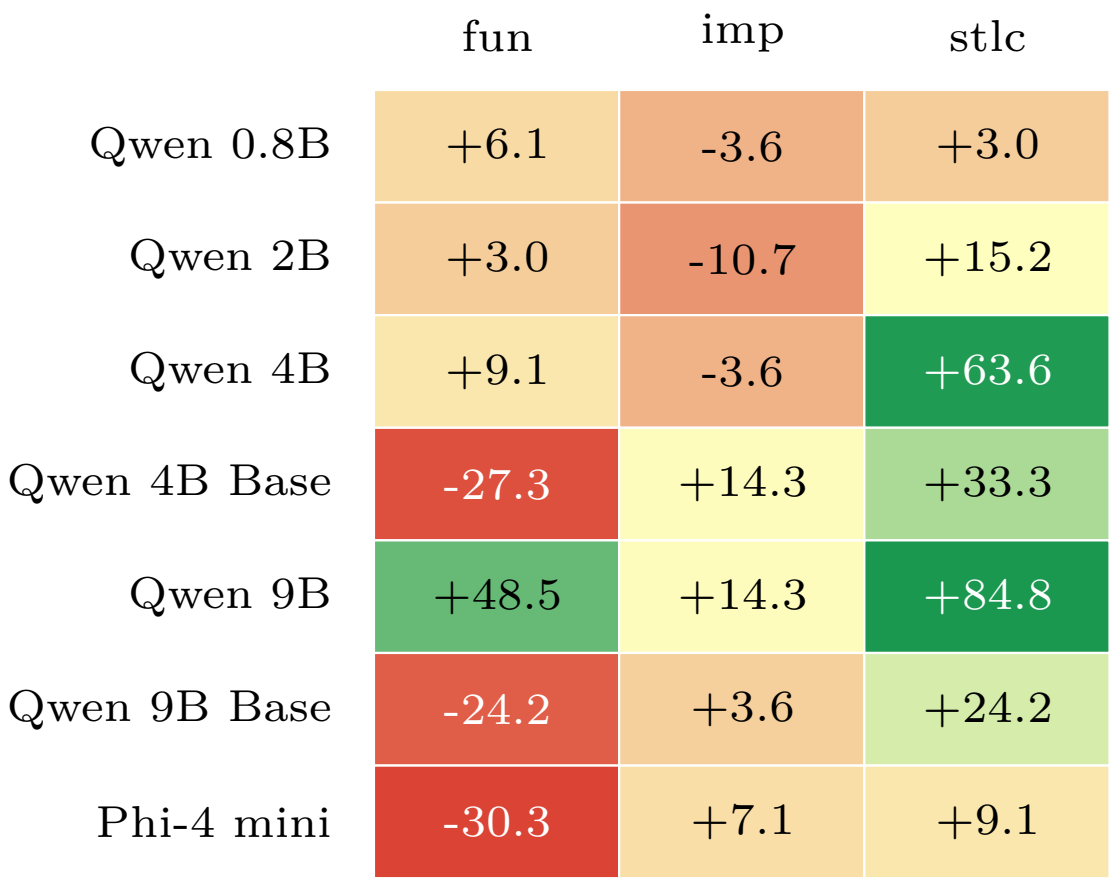

Fig. 8. Percentage point improvement between direct constrained generation and mixed mode

Fig. 9. State-transition trace for $\lambda f : B \to C.\lambda g : A \to B.\lambda x : A.f\,(g\,x)$.

Rule instance	Local effect	State after transition
SEED/CONSUMEREADY* Seed and scan the three nested lambda binders.	$\Omega_f[a \mapsto f, \tau \mapsto B \to C]$, $\Omega_g[a \mapsto g, \tau \mapsto A \to B]$, $\Omega_x[a \mapsto x, \tau \mapsto A]$ $\Gamma_3 = \Gamma_0[f : B \to C, g : A \to B, x : A]$	
PROCESSNONTERMINAL	Enter the body $f\,(g\,x)$ and seed the outer application.	$\eta_0 = \langle \text{Application}, p_{\mathsf{app}}, 0, [i, i), \Gamma_3, \Omega_0, \epsilon \rangle$ $\Omega_0 \vdash l : T_0 \to U_0, \quad r : T_0$
FINISH var / COMPLETERESUME	Finish f, then resume the outer application with f bound as l.	$\nu_f : B \to C$ $\mathsf{resolve}_N(\Omega_0, l, \nu_f) \quad \Rightarrow \quad T_0 = B,\ U_0 = C$
PROCESSNONTERMINAL	The right child r must have type B. Seed the inner application $g\,x$.	$\eta_1 = \langle \text{Application}, p_{\mathsf{app}}, 0, [j, j), \Gamma_3, \Omega_1, \epsilon \rangle$ $\Omega_1 \vdash l : T_1 \to U_1, \quad r : T_1$
FINISH var/ COMPLETERESUME	Finish g, then resume the inner application with g as binding l.	$\nu_g : A \to B$ $\mathsf{resolve}_N(\Omega_1, l, \nu_g) \quad \Rightarrow \quad T_1 = A,\ U_1 = B$
FINISH var	Finish x. It satisfies the right-child obligation of $g\,x$.	$\nu_x : A$ $\mathsf{resolve}_N(\Omega_1, r, \nu_x) \quad \Rightarrow \quad \Omega_1 \models r : A$
FINISH app	Finalize the inner application $g\,x$.	$\nu_{gx} : B$
COMPLETERESUME/ FINISH app	Resume the outer application with $g\,x$ as binding r, then finalize $f\,(g\,x)$.	$\mathsf{resolve}_N(\Omega_0, r, \nu_{gx}) \quad \Rightarrow \quad \Omega_0 \models r : B$ $\nu_{\text{body}} : C$
COMPLETERESUME/ FINISH lambda	Close the three lambda nodes and go back up	$\nu_{\lambda x} : A \to C$ $\nu_{\lambda g} : (A \to B) \to A \to C$ $\nu_{\lambda f} : (B \to C) \to (A \to B) \to A \to C$

```
// Identifier (supports Unicode)
Identifier ::= /[A-Za-z_][A-Za-z0-9]*/
TypeName ::= /[A-Za-z0-9_τ₁₂₃₄₅₆₇₈₉₀]+/
// Variables with var typing rule
Variable(var) ::= Identifier[x]
// Base types (parentheses are literals, hence quoted)
BaseType ::= TypeName | '(' Type ')'
// Function types (right-associative)
AtomicType ::= BaseType | '(' Type ')'
FunctionType ::= AtomicType '->' Type
Type ::= AtomicType | FunctionType
// Lambda abstraction (dot is a literal)
Lambda(lambda) ::= 'λ' Identifier[a] ':' Type[τ] '.' Expression[e]
// Atomic expressions
AtomicExpression ::= Variable | '(' Expression ')' | Lambda
// Applications (left-associative)
Application(app) ::= Expression[l] AtomicExpression[r]
// Expressions (start symbol)
Expression ::= AtomicExpression |  Application

// Typing Rules
x ∈ Γ
----------- (var)
Γ(x)

Γ[a:τ] ⊢ e : ?B
--------------------------- (lambda)
τ → ?B

Γ ⊢ l : ?A → ?B, Γ ⊢ r : ?A
--------------------------------- (app)
?B
```

Fig. 10. STLC grammar specification (`stlc.auf`).

```
Identifier ::= /[a-z][a-z0-9_]*/
TypeName ::= 'Int' | 'Bool'
BaseType ::= TypeName | '(' Type ')'
UnionType ::= BaseType '|' Type
Type ::= BaseType | UnionType
Integer(int_lit) ::= /[0-9]+/
Boolean(bool_lit) ::= 'true' | 'false'
Variable(var) ::= Identifier[x]
Group ::= '(' Expression ')'
AtomicExpr ::= Variable | Integer | Boolean | Group
ArithOp(arith_op) ::= Expression[lhs] ArithOperator[op] Expression[rhs]
ArithOperator ::= '+' | '-' | '*' | '/'
CompOp(comp_op) ::= AtomicExpr[lhs] CompOperator[op] AtomicExpr[rhs]
CompOperator ::= '==' | '≠' | '<' | '≤' | '>' | '≥'
Expression ::= ArithOp | CompOp | AtomicExpr
Declaration(decl) ::= 'let' Identifier[name] ':' Type[τ] '=' Expression[value] ';'
Assignment(assign) ::= Identifier[name] '=' Expression[value] ';'
IfStmt(if) ::= 'if' '(' Expression[cond] ')' Block[then_block]
IfElseStmt(if_else) ::= 'if' '(' Expression[cond] ')' Block[then_block] 'else' Block[else_block]
WhileStmt(while) ::= 'while' '(' Expression[cond] ')' Block[body]
Block(block) ::= '{' StatementList[stmts] '}'
Statement ::= Declaration | Assignment | IfStmt | IfElseStmt | WhileStmt
StatementList ::= Statement StatementList | Statement
// Start symbol
Program(program) ::= Block[main]

x ∈ Γ
----------- (var)
Γ(x)

----------- (int_lit)
'Int'

----------- (bool_lit)
'Bool'

Γ  ⊢ lhs : 'Int', Γ ⊢ rhs : 'Int'
------------------------------------------------- (arith_op)
'Int'

Γ  ⊢ lhs : 'Int', Γ ⊢ rhs : 'Int'
------------------------------------------------- (comp_op)
'Bool'

Γ  ⊢ value : τ
----------------------- (decl)
Γ → Γ[name:τ] ⊢ ∅

name ∈ Γ, Γ ⊢ value : Γ(name)
------------------------------ (assign)
∅

Γ ⊢ cond : 'Bool', Γ ⊢ then_block : ?T
--------------------------------------- (if)
?T

Γ ⊢ cond : 'Bool', Γ ⊢ then_block : ?T, Γ ⊢ else_block : ?T
------------------------------------------------------------ (if_else)
?T

Γ ⊢ cond : 'Bool', Γ ⊢ body : ?T
-------------------------------- (while)
?T
```

Fig. 11. Imp grammar specification (`imp.auf`).

```
Identifier ::= /[a-z][a-z0-9]*/
TypeName ::= /[A-Z][a-z0-9]*/

BaseType ::= TypeName | '(' Type ')'
FunctionType ::= BaseType '->' Type
Type ::= BaseType | FunctionType

Integer(int) ::= /[0-9]+/
Boolean(bool) ::= 'true' | 'false'
Float(float) ::= /[0-9]+\.[0-9]+/

IntOp ::= '+' | '-' | '*' | '/'
FloatOp ::= '+.' | '-.' | '*.' | '/.'

Variable(var) ::= Identifier[x]
Lambda(lambda) ::= '(' Identifier[param] ':' Type[τ] ')' '⇒' Expression[body]
Paren ::= '(' Expression ')'
Let(let) ::= 'let' Identifier[name] ':' Type[τ] '=' Expression[value] ';' Expression[body]
AtomicExpression ::= Variable | Integer | Float | Boolean | Lambda | Paren

PostfixTail ::= '(' Expression ')' PostfixTail | ε
PostfixExpression ::= AtomicExpression PostfixTail
IntBinary(bin_int) ::= Expression[left] IntOp[op] Expression[right]
FloatBinary(bin_float) ::= Expression[left] FloatOp[op] Expression[right]
Application(app) ::= AtomicExpression[func] '(' Expression[arg] ')'
                   | Application[func] '(' Expression[arg] ')'
// Top-level expression and start symbol
Expression ::= IntBinary | FloatBinary | Application | Let | AtomicExpression

x ∈ Γ
----------- (var)
Γ(x)

----------- (int)
'Int'

----------- (float)
'Float'

----------- (bool)
'Bool'

Γ[param:τ] ⊢ body : ?R
---------------------- (lambda)
τ -> ?R

Γ ⊢ value : τ, Γ[name:τ] ⊢ body : ?R
------------------------------------ (let)
?R

Γ ⊢ left : 'Int', Γ ⊢ right : 'Int'
---------------------------------- (bin_int)
'Int'

Γ ⊢ left : 'Float', Γ ⊢ right : 'Float'
-------------------------------------- (bin_float)
'Float'

Γ ⊢ func : ?A -> ?R, Γ ⊢ arg : ?A
--------------------------------- (app)
?R
```

Fig. 12. Fun grammar specification (`fun.auf`).